Malware Threats - Study Guide

Concepts, Analysis, Countermeasures & Lab Walkthroughs

Ethical Hacking / Network Security Study Notes
November 2026

1. Malware Concepts
2. Trojan Concepts
3. Virus and Worm Concepts
4. Malware Analysis
5. Countermeasures
6. Anti-Malware Software
7. Lab Note: RAT and Virus-Generator Exercises
8. Lab Walkthrough: Perform Online Malware Scanning Using VirusTotal
9. Lab Walkthrough: Perform a Strings Search Using BinText
10. Lab Walkthrough: Find Portable Executable Information Using PE Explorer
11. Lab Walkthrough: Perform Malware Disassembly Using IDA
12. Lab Walkthrough: Perform Port, Process, Startup Program, and Device Driver Monitoring

Quick Revision Summary

> ***Scope & Ethics Notice*** *This guide is for educational / certification-study purposes (e.g., CEH-style exam prep) and for authorized security work only. Any hands-on exercise below must be performed only in an isolated lab you own or control, or on systems you have explicit written permission to test. Creating, deploying, or distributing real malware against systems you do not own or lack authorization to test is illegal in most jurisdictions.* ***This guide does not include operational instructions for building or deploying real malware*** *(e.g., configuring a remote-access trojan or a virus-generation tool to infect a machine); those sections instead explain the concepts, what such a lab is meant to teach, and how defenders detect and respond to them.*

1. Malware Concepts

1.1 What is Malware?

Malware ("malicious software") is any program or code intentionally designed to damage, disrupt, gain unauthorized access to, or exfiltrate data from a computer system, without the owner's informed consent.

1.2 Common Techniques Attackers Use to Distribute Malware

- **Phishing / spear-phishing emails** with malicious attachments or links
- **Drive-by downloads** from compromised or malicious websites
- **Malvertising** (malicious online advertisements)
- **Removable media** (infected USB drives)
- **Software bundling / trojanized installers** (repackaged legitimate software)
- **Exploiting unpatched vulnerabilities** (browsers, OS, applications)
- **Social engineering** (fake software updates, cracked software, fake antivirus pop-ups)
- **Supply-chain compromise** (tampering with legitimate software update channels)

1.3 Components of Malware

Component	Purpose
Crypter	Hides malware from detection (encrypts/obfuscates the payload)
Downloader	Downloads additional malicious components post-infection
Dropper	Covertly installs the malware payload on the target
Exploit	Code that takes advantage of a vulnerability to deliver the payload
Injector	Injects malicious code into other running processes
Obfuscator	Conceals malware code via various techniques to evade analysis/detection
Packer	Compresses the malware binary to evade signature-based detection
Payload	The component that performs the intended malicious action
Malicious code	The actual instructions that cause the harmful behavior

1.4 Common Malware Categories

- **Trojan Horse** - disguises itself as legitimate software (see Section 2)
- **Virus** - self-replicates by attaching to host files/programs (see Section 3)
- **Worm** - self-replicates and spreads across networks without user action (see Section 3)
- **Ransomware** - encrypts victim data and demands payment for the decryption key
- **Spyware** - covertly monitors/records user activity (keystrokes, screenshots, credentials)

- **Adware** - displays unwanted advertisements, often bundled with other software
- **Rootkit** - hides the presence of malicious processes/files from the OS and security tools
- **Backdoor** - provides covert remote access bypassing normal authentication
- **Botnet / bot** - compromised machine (“zombie”) controlled remotely as part of a larger network
- **Fileless malware** - resides in memory/registry/scripts rather than as a file on disk, evading file-based AV scans
- **Logic bomb** - lies dormant until a specific condition/trigger is met

1.5 APT (Advanced Persistent Threat)

A prolonged, targeted attack in which an intruder gains and maintains unauthorized, stealthy access to a network, typically to continuously exfiltrate data rather than cause immediate damage. Characterized by: well-resourced and organized attackers, long dwell time, use of custom malware, and specific high-value targets.

1.6 Indicators of Compromise (IoC) Categories

- **Email indicators** (suspicious sender, subject, attachment hash)
- **Network indicators** (C2 domains/IPs, unusual DNS queries, beaconing patterns)
- **Host-based indicators** (new/unexpected files, registry keys, scheduled tasks, services)
- **Behavioral indicators** (process injection, privilege escalation attempts, unusual outbound connections)

2. Trojan Concepts

2.1 What is a Trojan?

A Trojan (Trojan Horse) is malware disguised as, or bundled with, a legitimate and desirable program. Unlike viruses/worms, a Trojan generally does **not self-replicate**; the victim is tricked into installing/running it, after which it opens the door for the attacker.

2.2 How Trojans Work (Typical Attack Flow)

1. Attacker creates/configures a Trojan (a **client** and a **server** component in classic RAT architecture).
2. The **server** (the piece installed on the victim) is wrapped/bound with a legitimate-looking file or lure (crack, pirated software, email attachment, fake update).
3. Victim executes the file; the server component silently installs, often establishing persistence (e.g., a registry Run key, a scheduled task, or a service).
4. The server "phones home" (beacons) to the attacker's **command-and-control (C2)** infrastructure, or opens a listening port awaiting connection.
5. The attacker's **client** application connects, giving remote control over the victim: file transfer, screen/webcam/microphone capture, keystroke logging, remote shell, etc.

2.3 Types of Trojans (by purpose)

Type	Purpose
Remote Access Trojan (RAT)	Gives the attacker full remote control of the victim machine
Backdoor Trojan	Opens a hidden entry point bypassing authentication
Botnet Trojan	Enlists the victim into a botnet for coordinated attacks (e.g., DDoS)
Banking Trojan	Targets online banking credentials/sessions
Downloader Trojan	Downloads and installs further malware
Proxy Trojan	Turns the victim into a proxy to relay attacker traffic
Data-Hiding / Ransom Trojan	Encrypts or hides victim data for extortion
Rootkit Trojan	Hides itself and other malware from detection
Mobile Trojan	Targets smartphones (SMS fraud, credential theft, spyware)
IoT Trojan	Targets embedded/IoT devices, often for botnet recruitment

2.4 Trojan Delivery and Evasion Techniques

- **Binding/wrapping**: joining the Trojan's installer with a legitimate executable so both run together.
- **Crypters/packers**: obfuscate or encrypt the binary so static, signature-based antivirus cannot recognize it.
- **Fake file extensions / double extensions** (e.g., `invoice.pdf.exe` with icon spoofing).
- **Process hollowing / injection**: running malicious code inside the memory space of a legitimate process to blend in.
- **Living-off-the-land**: abusing built-in OS tools (PowerShell, WMI, mshta) to avoid dropping new binaries.
- **Anti-VM / anti-debugging checks**: the Trojan detects sandbox/analysis environments and stays dormant to evade automated analysis.

2.5 Indicators of a Trojan Infection

- Unusually high CPU/network/disk usage, or new, unfamiliar processes
- Unexpected outbound connections to unknown hosts/ports
- New startup entries, services, or scheduled tasks not installed by the user
- Antivirus/firewall being silently disabled
- Unexplained file modifications, pop-ups, or system slowdowns

3. Virus and Worm Concepts

3.1 What is a Virus?

A computer virus is self-replicating code that **attaches itself to a host file or program** and requires some form of **user action** (opening a file, running a program) to execute and spread. It typically has: an infection mechanism, a trigger (condition for activation), and a payload (its damaging action).

3.2 Virus Lifecycle

1. **Design** - the virus is written/created.
2. **Replication** - it copies itself into other files/programs.
3. **Launch/Activation** - executed (by the user or trigger) and begins acting.
4. **Detection** - identified by AV vendors; signatures are created.
5. **Incorporation** - AV vendors update detection signatures/engines.
6. **Elimination** - infected systems are cleaned/patched; the strain declines.

3.3 Types of Viruses

Type	Description
File/Program Infector	Attaches to executable files
Boot Sector Virus	Infects the Master Boot Record, loads before the OS
Macro Virus	Embedded in document macros (e.g., Word/Excel)
Polymorphic Virus	Changes its own code/signature on each infection to evade detection
Metamorphic Virus	Fully rewrites its own code each time, more advanced than polymorphic
Multipartite Virus	Infects both boot sectors and files
Stealth/Tunneling Virus	Hides changes and evades antivirus by intercepting system calls
Cavity Virus	Hides in unused space within a host file, keeping file size unchanged
Encrypted Virus	Encrypts its payload, decrypting only at execution time
Sparse Infector	Infects only occasionally/selectively to reduce detectability
Companion Virus	Creates a malicious file with a name/priority that runs instead of the legitimate one

3.4 What is a Worm?

A worm is self-replicating malware that spreads **independently across networks**, exploiting vulnerabilities or weak credentials, **without requiring a host file or user action**. Worms often cause damage purely through network/resource consumption (bandwidth, CPU) in addition to any payload they carry.

3.5 Virus vs. Worm

Aspect	Virus	Worm
Needs a host file	Yes	No
Needs user action to spread	Yes	No (self-propagating)
Spreads over network automatically	Usually no	Yes
Typical vector	Infected files/documents	Network vulnerabilities, email, removable media

3.6 Why Attackers Create Viruses/Worms

Financial gain, intellectual challenge/notoriety, corporate or state-sponsored sabotage, cyberwarfare, extortion (ransomware-style payloads), and to distribute other malware.

4. Malware Analysis

4.1 What is Malware Analysis?

The process of studying a malware sample's structure, functionality, and behavior to understand its purpose, how it operates, how to detect it, and how to remediate/contain an infection.

4.2 Types of Malware Analysis

Static Analysis (without executing the sample)

- File fingerprinting (hashing: MD5/SHA1/SHA256) for identification and threat-intel lookups
- String extraction (readable strings often reveal URLs, IPs, file paths, commands)
- Identifying file-format/PE headers, imported/exported functions, sections
- Disassembly (converting machine code to assembly for manual review)
- Scanning with multiple antivirus engines / threat-intel platforms

Dynamic (Behavioral) Analysis (executing the sample in a controlled environment)

- Monitoring process, registry, file system, and network activity in a sandbox
- Observing persistence mechanisms created (services, scheduled tasks, Run keys)
- Monitoring outbound C2 communications
- API/system call tracing (e.g., with tools like Process Monitor, API Monitor)

> ***Best practice:*** *Malware analysis is performed only in an* ***isolated environment*** *- an air-gapped or host-only-networked virtual machine with no production network access and a clean snapshot to revert to afterward.*

4.3 The Sandbox Approach

A sandbox is an isolated, instrumented execution environment (often a disposable VM) used to safely run and observe malware behavior. Examples: **Cuckoo Sandbox**, **Any.Run**, **Joe Sandbox**, **Hybrid Analysis**.

4.4 Typical Malware Analysis Workflow

1. Acquire the sample safely (isolated, read-only handling; keep original hash).
2. Perform static analysis (hash, strings, PE/file-format inspection).
3. Submit the hash/sample to reputation and multi-engine scanning services (e.g., VirusTotal).
4. If novel/unknown, perform dynamic analysis in a sandboxed VM.
5. If deeper understanding is needed, disassemble/decompile and reverse-engineer key functions.
6. Document indicators of compromise (hashes, IPs/domains, mutexes, registry keys, file paths).
7. Produce detection signatures (YARA rules, IDS/IPS signatures) and a remediation/response plan.

5. Countermeasures

5.1 Trojan Countermeasures

- Avoid downloading/executing software from untrusted sources; verify digital signatures.
- Keep OS and applications patched to close exploited vulnerabilities.
- Use application allow-listing to block unauthorized executables.
- Monitor outbound network connections for unexpected destinations/ports.
- Disable unnecessary services and close unused ports.
- Educate users on phishing and social-engineering recognition.
- Deploy EDR (Endpoint Detection & Response) for behavioral detection beyond signatures.

5.2 Virus and Worm Countermeasures

- Keep antivirus/anti-malware definitions and engines up to date.
- Patch management to close the vulnerabilities worms exploit for network propagation.
- Disable macros by default in Office documents (or restrict to signed macros).
- Network segmentation to limit worm propagation across subnets.
- Email filtering/attachment sandboxing at the gateway.
- Regular, tested, offline backups to recover from destructive payloads.
- Principle of least privilege to limit what an infected account/process can do.

5.3 General Malware Defense-in-Depth

1. **Perimeter controls**: firewalls, secure email/web gateways, IDS/IPS.
2. **Endpoint controls**: antivirus/EDR, host firewalls, application allow-listing.
3. **Identity controls**: MFA, least privilege, strong password policy.
4. **Patch & configuration management**: timely patching, hardened baselines.
5. **User awareness training**: phishing simulations, safe-browsing practices.
6. **Monitoring & response**: SIEM correlation, IoC-based detection, incident-response playbooks.
7. **Backup & recovery**: offline/immutable backups, tested restore procedures.

6. Anti-Malware Software

6.1 How Antivirus/Anti-Malware Software Detects Threats

Detection Method	Description
Signature-based	Matches file hashes/byte patterns against a database of known malware signatures. Fast but ineffective against new/polymorphic malware.
Heuristic-based	Looks for suspicious code characteristics/structure that resemble malware behavior, even without an exact signature match.
Behavior-based	Monitors what a program actually *does* at runtime (file/registry/network activity) and flags malicious patterns.
Cloud/reputation-based	Checks file hash/reputation against a cloud database of known-good and known-bad files in near real time.
Machine-learning-based	Uses trained models on large datasets of malicious/benign samples to classify previously unseen files.
Sandboxing	Detonates suspicious files in an isolated environment before allowing them to run on the real endpoint.

6.2 Features to Look for in Anti-Malware Software

- Real-time/on-access scanning
- Scheduled full-system scans
- Automatic signature/engine updates
- Web/email protection (phishing and malicious-link filtering)
- Firewall integration
- Ransomware-specific protection (e.g., controlled folder access)
- Quarantine and rollback capability
- Centralized management/reporting for enterprise deployment (EDR/XDR platforms)

6.3 Examples of Anti-Malware Solutions

Consumer/Enterprise AV: Microsoft Defender, Kaspersky, Bitdefender, Norton, ESET. Enterprise EDR/XDR: CrowdStrike Falcon, Microsoft Defender for Endpoint, SentinelOne, Carbon Black.

6.4 Limitations of Anti-Malware Software

- Signature-based engines lag behind brand-new (“zero-day”) malware.
- Fileless and living-off-the-land techniques can evade file-based scanning.
- Heavily obfuscated/packed malware can sometimes evade heuristic engines.
- Antivirus itself can be a target (attackers attempt to disable/bypass it) — defense-in-depth remains essential.

7. Lab Note: RAT and Virus-Generator Exercises

The original lab list for this module typically includes two hands-on exercises using real, functioning malware tools: using the **njRAT** remote-access trojan to take control of a victim machine, and using the **JPS Virus Maker** toolkit to build and deploy a virus. These are not included here as operational walkthroughs.

Why: njRAT is a real, actively-used remote-access trojan, and JPS Virus Maker is a dedicated malware-construction toolkit — not general-purpose security tools. Step-by-step instructions for configuring and deploying either one would amount to a functional guide for compromising a machine, regardless of the "lab" framing, so that falls outside what this guide can provide.

7.1 What These Labs Are Meant to Teach

In a certification course, these exercises exist to make two things concrete:

- **The RAT lab** demonstrates the Section 2.2 attack flow end to end: building a server/client pair, binding the server to a lure file, achieving persistence on execution, and the attacker's client connecting back for remote control (file access, keylogging, screen/webcam capture, shell access). The practical lesson is *how little user interaction a convincing lure requires*, and why endpoint monitoring (Section 7.6 below) and user-awareness training are essential.
- **The virus-maker lab** demonstrates the Section 3.3 virus lifecycle: a builder tool lets a would-be author pick a payload and infection behavior (e.g., file infection, startup persistence, disabling Task Manager/Registry Editor, fake error messages) without writing code, and bind it to a decoy filename/icon. The practical lesson is *how low the technical barrier to entry for commodity malware has become*, and why signature-based AV alone is insufficient (Section 6.4).

7.2 What to Do Instead if You Need Hands-On Practice

- Use **deliberately-vulnerable, pre-built lab malware samples** designed for safe analysis practice (e.g., samples from **theZoo**, **Malware-Traffic-Analysis.net** pcaps, or vendor-provided CTF/training ranges), analyzed only in an isolated sandbox — never executed on a production or internet-connected host.
- Practice the **detection and analysis side** fully using Sections 8–12 below (VirusTotal, BinText, PE Explorer, IDA, and system monitoring) against known/public sample hashes and public analysis reports rather than live binaries.
- For a guided, safe equivalent of the RAT lab, use a **benign remote-access tool you control** (e.g., a self-hosted reverse-shell exercise in a CTF range such as TryHackMe/HackTheBox) to study the same connect-back/C2 concepts without using real malware.

7.3 Recognizing a RAT Infection (Blue-Team View)

- Unrecognized process names, or legitimate-looking processes running from unusual paths (e.g., `svchost.exe` outside `System32`)
- A persistent outbound connection to an unfamiliar IP/domain, often on an unusual or repeatedly-changing port
- New registry Run keys, scheduled tasks, or services the user didn't create
- Webcam/microphone activity indicators activating without a corresponding user action
- Antivirus/firewall settings silently changed or disabled

7.4 Recognizing a Virus/Worm Infection (Blue-Team View)

- Rapid, unexplained growth in file sizes or a burst of newly-modified executables
- Task Manager, Registry Editor, or Control Panel access disabled unexpectedly
- Unusual outbound network scanning activity from a host (worm propagation behavior)
- Multiple endpoints on the same network exhibiting identical new symptoms within a short time window

7.5 Safe, Legitimate Hands-On Alternative: Build and Detect a Harmless Test File

To rehearse the *detection* workflow without any real malware, security training commonly uses the **EICAR test file** — a standardized, completely harmless text string that every major antivirus engine is designed to flag as if it were malware, purely to test that AV/EDR tooling is active and responding correctly:

```
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
```

Save that exact string to a `.txt` or `.com` file in your lab environment and confirm your antivirus immediately quarantines/flags it — this is the industry-standard, zero-risk way to validate that endpoint protection is working, and a reasonable stand-in for “infect the target machine and confirm detection” without using live malware.

7.6 Monitoring Controls That Catch Both RATs and Viruses

The system-monitoring techniques in Section 12 (ports, processes, startup programs, device drivers) are precisely the controls that reveal the artifacts both lab types are meant to teach you to produce — practicing Section 12 thoroughly covers the detection half of both exercises.

8. Lab Walkthrough: Perform Online Malware Scanning Using VirusTotal

8.1 Objective

Use a legitimate, free multi-engine scanning service to check a suspicious file or hash against 70+ antivirus engines and threat-intelligence sources.

8.2 Steps

1. Navigate to **https://www.virustotal.com** in a browser.
2. Choose one of three submission methods:
   - **File**: Click **Choose file**, select the suspicious file, and upload it (note: uploaded files become visible to the VirusTotal community/vendors, so never upload files containing sensitive/confidential data).
   - **URL**: Paste a suspicious URL to check its reputation.
   - **Search**: Paste a known file hash (MD5/SHA1/SHA256) to check prior scan results **without uploading the file itself** — the safer option when you already have a hash and want to avoid exposing the sample.
3. Review the **Detection** tab: shows how many of the participating engines flag the file as malicious, and under what family/name each vendor classifies it.
4. Review the **Details** tab: file type, size, hashes, compilation timestamp, and any digital signature information.
5. Review the **Relations** tab: related IPs/domains contacted, other files that drop/are dropped by this one, and similar samples.
6. Review the **Behavior** tab (if a sandbox report exists): observed registry/file/network activity from automated dynamic analysis.
7. Record the **hash and detection ratio** as part of your incident documentation / IoC list.

8.3 Expected Result

A clear verdict on how many engines flag the sample, which malware family it’s associated with (if known), and any related infrastructure (C2 domains/IPs) — directly supporting the IoC documentation step in Section 4.4.

8.4 Caveats

- A “0 detections” result does not guarantee a file is safe — it may be brand new or evasive; combine with behavioral/sandbox analysis.
- Treat all uploaded results as effectively public; never upload sensitive internal documents even if you suspect they’re infected — hash-only lookups avoid this risk.

9. Lab Walkthrough: Perform a Strings Search Using BinText

9.1 Objective

Extract human-readable ASCII/Unicode strings embedded in a binary to quickly surface clues such as URLs, IP addresses, file paths, registry keys, or error messages — a core static-analysis technique.

9.2 Steps

1. Launch **BinText** and click **Browse** to select the target executable/DLL.
2. Set the **Advanced View** options as needed: enable **ASCII**, **Unicode**, and **Resource** string types for a complete extraction pass.
3. Click **Go** to run the scan; BinText lists every matching string along with its **file offset** and **memory address**.
4. Use the **Find** box to search for interesting keywords, e.g.:
   - `http://` / `https://` (possible C2 URLs)
   - `.dll`, `.exe` (dependent/dropped files)
   - `cmd.exe`, `powershell` (command execution indicators)
   - `HKEY_` (registry keys touched, e.g., persistence locations)
   - `SOFTWARE\Microsoft\Windows\CurrentVersion\Run` (classic persistence key)
5. Export the string list (**Save As**) for inclusion in your analysis report.

9.3 Equivalent Cross-Platform Tools

On Linux/macOS, the built-in `strings` utility achieves the same goal:

```
strings -a suspicious.exe | grep -Ei "http|\.dll|cmd\.exe|HKEY_"
```

9.4 Expected Result

A list of plaintext artifacts (URLs, file paths, registry keys) that give an immediate, low-effort first read on the sample’s likely purpose before deeper disassembly is attempted.

10. Lab Walkthrough: Find Portable Executable Information Using PE Explorer

10.1 Objective

Inspect a Windows PE (Portable Executable) file's internal structure — headers, sections, imports/exports — to understand its capabilities and compilation characteristics.

10.2 Steps

1. Open **PE Explorer** and load the target `.exe` / `.dll` file.
2. Review the **File Header** panel: machine type (x86/x64), number of sections, timestamp (compile date — often forged but still informative), and characteristics flags (e.g., whether it's a DLL, whether it's stripped of debug info).
3. Review the **Optional Header**: entry point address, image base, subsystem (GUI/console/driver), and linker version.
4. Open the **Section Viewer**: lists sections such as `.text` (code), `.data`, `.rdata`, `.rsrc` (resources), with their virtual/raw sizes. An unusually high-entropy or oddly-named/extra section can indicate **packing** or appended malicious data.
5. Open the **Import Table Viewer**: lists every DLL and function the binary imports — a key behavioral indicator. For example:
   - `WININET.dll` / `WS2_32.dll` imports → network capability
   - `ADVAPI32.dll` (`RegSetValueEx`, `RegCreateKeyEx`) → registry manipulation (persistence)
   - `KERNEL32.dll` (`CreateRemoteThread`, `WriteProcessMemory`, `VirtualAllocEx`) → classic process-injection API combination
   - `USER32.dll` (`SetWindowsHookEx`) → potential keylogging
6. Open the **Export Table Viewer** (for DLLs) to see what functions the file exposes to other code.
7. Use the built-in **Resource Viewer** to inspect embedded icons, dialogs, or strings, and the **Dependency Scanner** to map the full chain of DLL dependencies.
8. Optionally run the bundled **disassembler view** for a quick look at the entry-point code before moving to a dedicated tool like IDA (Section 11).

10.3 Expected Result

A structural profile of the binary — its imports in particular — that lets you infer likely capability (network communication, persistence, injection, keylogging) purely from static metadata, without executing the file.

11. Lab Walkthrough: Perform Malware Disassembly Using IDA

11.1 Objective

Use IDA (Interactive DisAssembler) to convert machine code into human-readable assembly and begin reverse-engineering a sample's logic.

11.2 Steps

1. Open **IDA Free/Pro** and load the target binary (**File → New**, select the file). IDA runs its **auto-analysis** pass, identifying functions, strings, and cross-references.
2. Review the **Functions window**: lists every function IDA identified, named (e.g., `sub_401000`) or recognized from known library signatures (**FLIRT** signatures match statically-linked library code so you can skip analyzing it).
3. Open the **Strings window** (`Shift+F12`) to cross-reference the same artifacts found with BinText (Section 9), but now with the ability to jump directly to the code location that references each string (right-click → **Jump to xref**).
4. Switch to **Graph view** on the entry-point or `main` function to visually trace the program's branching logic (conditional jumps, loops) — often easier to follow than linear assembly.
5. Use the **Imports window** to cross-reference the same Windows API calls found in PE Explorer (Section 10), and for each suspicious call (e.g., `CreateRemoteThread`), use **Jump to xref** to see exactly where/how it's used in context.
6. Rename variables/functions as you understand them (e.g., `sub_401000` → `decrypt_payload`) to build a readable map of the sample's logic as you go.
7. For packed/obfuscated samples, identify the **unpacking stub** (a tight loop that decodes a data block, followed by a jump to the now-decoded code) and consider dynamic analysis (a debugger, dumping memory post-unpacking) rather than attempting full static disassembly of packed bytes.
8. Document key findings: the C2 communication routine, persistence routine, encryption/obfuscation routine, and any anti-analysis checks (VM/debugger detection).

11.3 Expected Result

A readable map of the sample's core logic — how it establishes persistence, communicates outward, and (if present) attempts to evade analysis — feeding directly into the IoC and detection-signature steps of Section 4.4.

11.4 Note on Scope

Full reverse engineering is a specialist skill; for most incident-response purposes, static (Sections 9–10) plus sandboxed dynamic analysis (a safe automated sandbox report) answers the "what does this do" question faster than manual disassembly, which is best reserved for samples that evade sandboxes or require deep understanding (e.g., for signature/YARA-rule writing).

12. Lab Walkthrough: Perform Port, Process, Startup Program, and Device Driver Monitoring

12.1 Objective

Use built-in and free Windows utilities (primarily the **Sysinternals Suite**) to baseline a clean system and then identify anomalies that indicate malware presence — the core "blue-team" skill referenced throughout this guide.

12.2 Port Monitoring

Goal: Identify unexpected listening ports or active connections to unfamiliar remote hosts.

```
netstat -ano          # list all connections/listeners with owning PID
netstat -anob         # (admin) also shows owning executable name
```

Or use **TCPView** (Sysinternals GUI equivalent): launch it, sort by **State** or **Remote Address**, and investigate any `ESTABLISHED` connection to an unfamiliar IP, especially on non-standard ports, or any process you don't recognize that is listening.

Red flags: a process with no clear purpose holding an outbound connection to a foreign IP on an odd port; a listener on a high port you didn't open; repeated reconnect attempts to the same external address (beaconing).

12.3 Process Monitoring

Goal: Spot processes with suspicious names, paths, parent/child relationships, or resource usage.

Use **Process Explorer** (Sysinternals): 1. Launch as Administrator; enable **View → Lower Pane → DLLs** to inspect loaded modules per process. 2. Check the **Image** tab of a suspicious process for its on-disk **path** — malware frequently impersonates legitimate names (`svchost.exe`, `explorer.exe`) while running from the wrong directory (e.g., `%TEMP%` or `%APPDATA%` instead of `System32`). 3. Right-click → **Check VirusTotal** (Process Explorer has a built-in hash lookup against VirusTotal) for an instant reputation check on any running image. 4. Review the **parent process** — e.g., Word (`WINWORD.exe`) spawning `powershell.exe` or `cmd.exe` is a classic macro-malware indicator. 5. Use **Verify Signer** to check whether the binary is digitally signed by a trusted publisher.

12.4 Startup Program Monitoring

Goal: Find persistence mechanisms malware installs to survive a reboot.

Use **Autoruns** (Sysinternals) — the most comprehensive tool for this: 1. Launch as Administrator; it enumerates **every** autostart location: Registry Run/RunOnce keys, Startup folder, Scheduled Tasks, Services, Browser Helper Objects/extensions, Winlogon entries, and more, across all tabs. 2. Use **Options → Hide Microsoft Entries** to filter out well-known, signed Windows components and surface third-party/unknown entries. 3. Investigate any unsigned entry, or an entry pointing to a file in an unusual location (`%TEMP%`, `%APPDATA%`, a randomly-named folder). 4. Right-click an entry → **Check VirusTotal** for an immediate reputation check, or **Jump to Entry/Image** to inspect the file directly. 5. Disable (uncheck) or delete confirmed-malicious entries, then reboot and re-scan to confirm persistence is removed.

Lightweight CLI alternative:

```
reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run
reg query HKLM\Software\Microsoft\Windows\CurrentVersion\Run
schtasks /query /fo LIST /v
```

12.5 Device Driver Monitoring

Goal: Detect malicious or rootkit-style kernel-mode drivers, which operate with the highest privilege level and can hide other artifacts.

1. In **Autoruns**, check the **Drivers** tab specifically — look for unsigned drivers or drivers loading from non-standard paths.
2. Use the built-in **driverquery** command for a quick inventory:

```
driverquery /v /fo table
driverquery /si      # shows signed vs. unsigned drivers specifically
```

3. Cross-reference any unfamiliar driver name against VirusTotal or the vendor's site.
4. For deeper rootkit-hunting, specialized tools (e.g., **GMER**, **TDSSKiller**) compare the kernel's view of the system against a lower-level scan to reveal hidden drivers/processes that standard tools can't see — a hallmark of rootkit concealment.

12.6 Putting It Together: A Baseline-and-Compare Workflow

1. On a known-clean lab VM, take a snapshot and run Autoruns, Process Explorer, TCPView, and `driverquery` — save the output as your **baseline**.
2. After deliberately running a known-safe test artifact (e.g., the EICAR file from Section 7.5) or simply after normal use over time, re-run the same tools.
3. **Diff** the two captures to surface anything new: an unfamiliar process, a new Run key, a new listening port, or a new driver.
4. Investigate every new item with a hash lookup (VirusTotal), and if still ambiguous, escalate to the static/dynamic analysis techniques in Sections 9–11.

12.7 Expected Result

A repeatable, tool-supported method for spotting the exact artifacts that RATs, viruses, and worms leave behind (Sections 7.3–7.4), without needing to execute or possess the malware itself.

Quick Revision Summary

#	Topic	One-line takeaway
1	Malware Concepts	Broad umbrella term; know the components (dropper, packer, payload) and categories.
2	Trojan Concepts	Disguised, non-self-replicating; relies on the victim running it (server/client RAT model).
3	Virus and Worm Concepts	Virus needs a host file + user action; worm self-propagates over networks.
4	Malware Analysis	Static (no execution) then dynamic (sandboxed execution); always isolate the sample.
5	Countermeasures	Patch, least privilege, segmentation, awareness training, tested backups.
6	Anti-Malware Software	Signature + heuristic + behavior + ML + cloud reputation, layered together.
7	RAT/Virus-Maker Labs	Conceptual only here; use EICAR and CTF ranges for safe hands-on practice.
8	VirusTotal	Multi-engine hash/file/URL reputation check; prefer hash lookups over uploads.
9	BinText / strings	Fast static win: pull URLs, paths, registry keys straight out of the binary.
10	PE Explorer	Imports table reveals likely capability (network, registry, injection, keylogging).
11	IDA Disassembly	Converts code to assembly; trace C2/persistence routines and rename as you learn.
12	System Monitoring	Autoruns + Process Explorer + TCPView + driverquery = the practical detection toolkit.